

PATATA CALIENTE EN SERVIDORES TCP Y UDP

Programa de sockets en C

DESCRIPCIÓN BREVE
Programa de sockets que usará el
UDP para comunicar a un servido
tal manera que el cliente juegue a

Unai García Blanco – 709159
Jorge Caballero González - 7
2023-24

Introducción:

En esta práctica el objetivo es crear u programa de sockets en C en el que, mediante mensajes por protocolo TCP o UDP, un servidor y un determinado cliente se comunicaran para que el cliente juegue a una versión de preguntas y respuestas del conocido juego “La Patata Caliente”.

Los protocolos que se han usado en esta implementación, como ya se ha dicho, han sido TCP y UDP y ha habido algunas diferencias de un protocolo al otro.

1. **Conexión vs. Sin conexión:**
    - **TCP:** Proporciona una conexión orientada. Antes de intercambiar datos, se establece una conexión entre el emisor y el receptor.
    - **UDP:** Ofrece una comunicación sin conexión. Los datos se envían como datagramas individuales sin establecer una conexión previa.
2. **Fiabilidad:**
    - **TCP:** Garantiza la entrega de datos y reconoce la recepción. Se encarga de retransmitir los datos perdidos.
    - **UDP:** No garantiza la entrega ni realiza retransmisiones. Algunos datos pueden perderse sin aviso.
3. **Orden de entrega:**
    - **TCP:** Asegura que los datos lleguen en el orden correcto.
    - **UDP:** No garantiza el orden de entrega. Los datagramas pueden llegar en un orden diferente al enviado.
4. **Aplicaciones típicas:**
    - **TCP:** Utilizado en aplicaciones que requieren entrega confiable y control de flujo, como navegadores web, transferencia de archivos y correo electrónico.
    - **UDP:** Adecuado para aplicaciones en tiempo real que pueden tolerar pérdida de datos, como transmisiones en vivo, juegos en línea y videollamadas.
5. **Ejemplos de uso:**
    - **TCP:** Conexiones seguras donde la integridad de los datos es crítica.
    - **UDP:** Aplicaciones que priorizan la velocidad y la transmisión inmediata sobre la fiabilidad.

En resumen, la elección entre TCP y UDP depende de los requisitos específicos de la aplicación. Entonces, para nuestro programa, lo más idóneo sería usar TCP ya que nos garantiza la conexión, el orden de llegada de los mensajes, etc.

PROTOCOLO TCP:

Como ya hemos comentado, TCP en este caso, es el protocolo más apropiado para realizar el objetivo de enviar mensajes, parecido al servicio SMTP estudiado en clase.

El proceso que se ha seguido para la implementación es el siguiente:

- Se crea un socket en el servidor, y posteriormente se creará para cada cliente que solicita la conexión, un proceso hijo y un socket para ese proceso hijo, por lo que cada cliente tendrá un proceso hijo y un socket exclusivo para él.
- Posteriormente, la lógica que se ha seguido es que, a raíz de la trama que envía el servidor con código 220, el cliente irá enviando tramas de tipo char con un determinado mensaje para simular la comunicación entre este y el servidor. Las tramas las obtendrá el cliente desde ordenes.txt y el servidor las procesará según la respuesta que se espera, además de procurando que todas las tramas que envía y recibe, terminan en \r\n.
- Para finalizar la ejecución del cliente, este enviará una trama con contenido ADIOS y la comunicación terminará

Un ejemplo de ejecución es el siguiente:

- Se realiza la compilación del servidor y el cliente
- Se ejecutan con los respectivos comandos
- La ejecución nos mostrará por consola si ha salido todo como esperábamos y nos dará como resultado dos ficheros, nºpuertoTCP.txt un registro del cliente y peticionesTCP.log como registro del intercambio de mensajes.

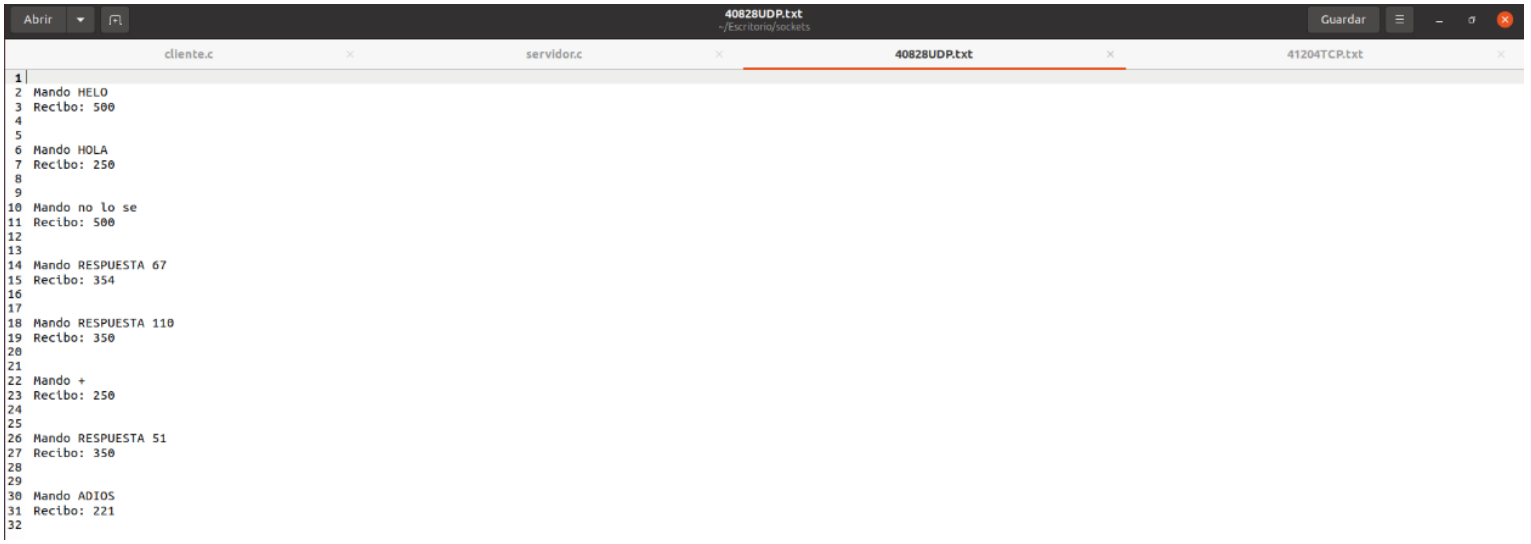
Ejemplo de make y ejecución:

```
unaigb@unaigb-VirtualBox:~/Escritorio/sockets$ make servidor
gcc     -c -o servidor.o servidor.c
gcc  -o servidor servidor.o
unaigb@unaigb-VirtualBox:~/Escritorio/sockets$ make cliente
gcc     -c -o cliente.o cliente.c
gcc  -o cliente cliente.o
unaigb@unaigb-VirtualBox:~/Escritorio/sockets$ ./servidor
unaigb@unaigb-VirtualBox:~/Escritorio/sockets$ ./cliente localhost TCP ordenes.txt
(C-TCP)Connected to localhost on port 40220 at Tue Dec 12 18:28:20 2023
(S-TCP)Startup from localhost, port 40220, IP 16777343, Protocol TCP, at Tue Dec 12 18:28:20 2023
(C-TCP)All done at Tue Dec 12 18:28:20 2023
unaigb@unaigb-VirtualBox:~/Escritorio/sockets$ (S-TCP)Completed localhost port 40220, 1 requests, IP 16777343, Protocol TCP, at Tue Dec 12 18:28:21 2023
```

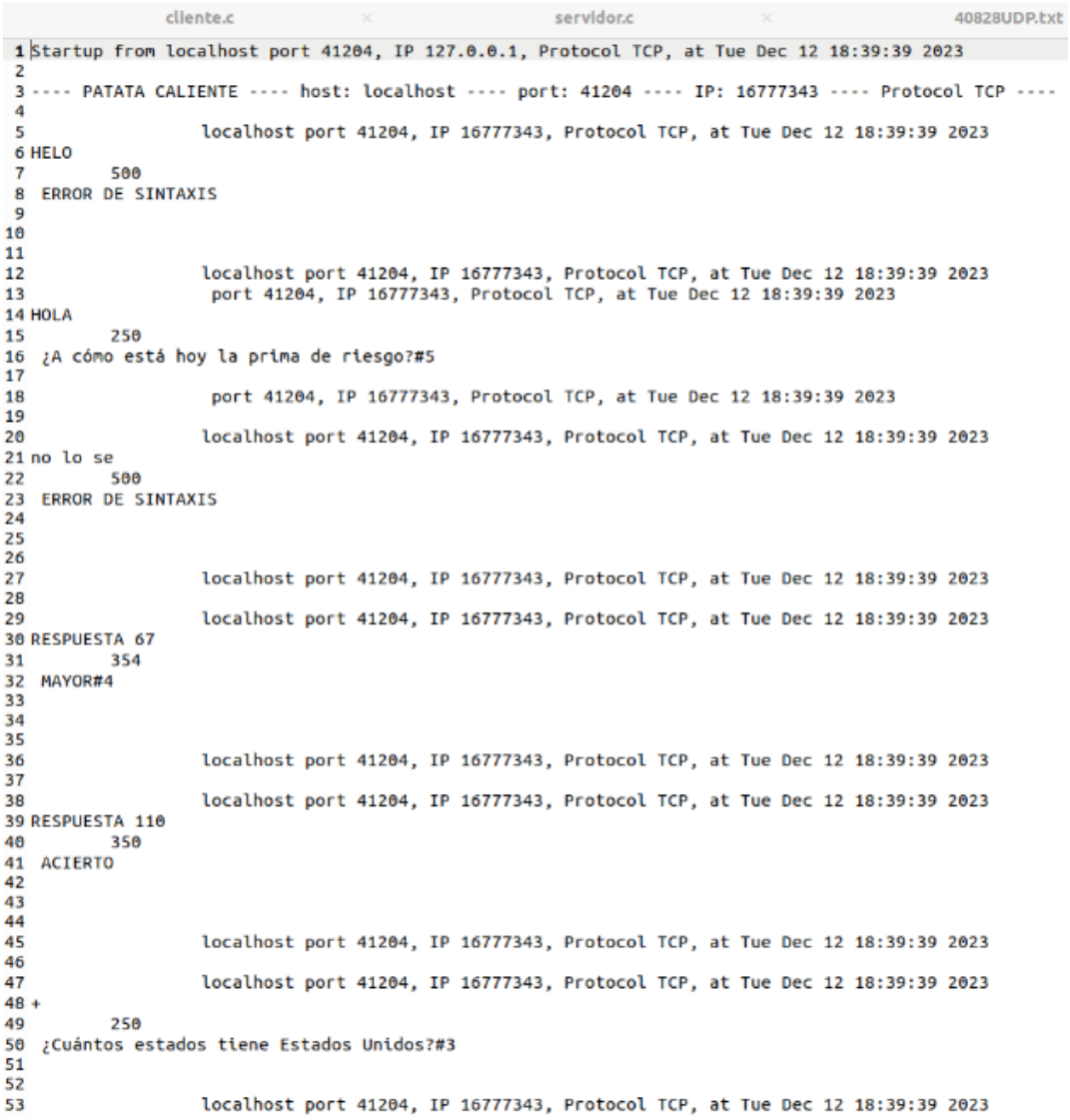
Ejemplo de cómo acabar el proceso servidor una vez queramos cerrar completamente el programa:

```
unaigb@unaigb-VirtualBox:~/Escritorio/sockets$ ps
    PID TTY          TIME CMD
   1862 pts/0    00:00:00 bash
   1972 pts/0    00:00:00 servidor
   2024 pts/0    00:00:00 ps
unaigb@unaigb-VirtualBox:~/Escritorio/sockets$ kill 1972
unaigb@unaigb-VirtualBox:~/Escritorio/sockets$
Fin de programa servidor!
```

Ejemplo de nºpuertoTCP.txt (Registro del cliente):



Ejemplo de peticionesTCP.log:



Continuación:

```
54
55                  localhost port 41204, IP 16777343, Protocol TCP, at Tue Dec 12 18:39:39 2023
56 RESPUESTA 51
57          350
58  ACIERTO
59
60
61
62                  localhost port 41204, IP 16777343, Protocol TCP, at Tue Dec 12 18:39:39 2023
63
64                  localhost port 41204, IP 16777343, Protocol TCP, at Tue Dec 12 18:39:39 2023
65
66 ADIOS
67          221
68  Cerrando el servicio
69                  localhost port 41204, IP 16777343, Protocol TCP, at Tue Dec 12 18:39:39 2023
70 Completed localhost port 41204, 1 requests, IP 16777343, Protocol TCP, at Tue Dec 12 18:39:40 2023
```

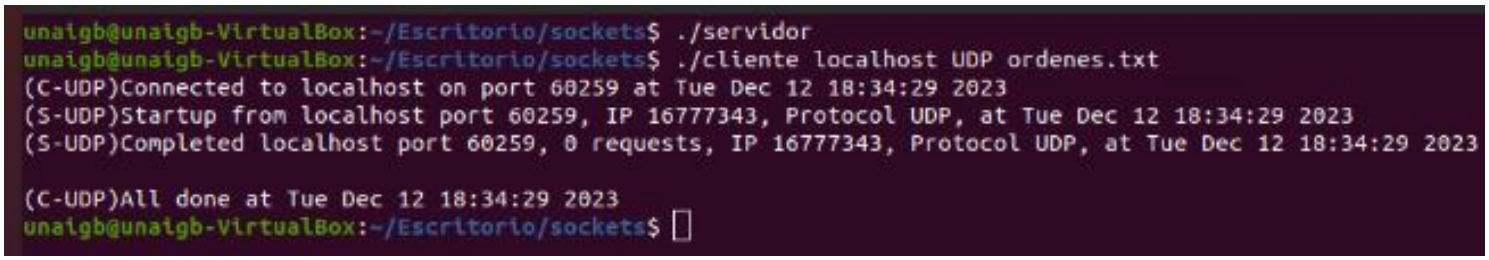
PROTOCOLO UDP:

Este protocolo presenta una gran cantidad de limitaciones respecto a TCP como ya se ha especificado anteriormente a la hora de implementar la comunicación mediante mensajes. Esto se debe a que a la hora de crear un nuevo socket no se puede utilizar la función accept, por lo que optamos por solucionar esa deficiencia con un fork que cree un proceso hijo y que este proceso hijo cree un nuevo socket para realizar la comunicación con el cliente que tienen asignado.

Para saber que un cliente requiere de un nuevo medio de comunicación, este enviará un mensaje de inicio de comunicación y cuando este llega y el servidor lo procesa, se lleva a cabo el proceso explicado anteriormente.

La ejecución del programa es igual que en TCP, por lo que se mostrarán las imágenes de la ejecución.

Ejemplo de make y ejecución:



En este caso, la forma de acabar con el proceso es la misma:



Ejemplo de nºpuertoUDP.txt (Registro del cliente):

cliente.c | servidor.c | 40828UDP.txt

```
1
2  Mando HELO
3  Recibo: 500
4
5
6  Mando HOLA
7  Recibo: 250
8
9
10 Mando no lo se
11 Recibo: 500
12
13
14 Mando RESPUESTA 67
15 Recibo: 354
16
17
18 Mando RESPUESTA 110
19 Recibo: 350
20
21
22 Mando +
23 Recibo: 250
24
25
26 Mando RESPUESTA 51
27 Recibo: 350
28
29
30 Mando ADIOS
31 Recibo: 221
32
```

Ejemplo de peticionesUDP.log:

cliente.c | servidor.c | 40828UDP.txt

```
1  Startup from localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
2
3  ---- PATATA CALIENTE ---- host: localhost ---- port: 40828 ---- IP: 16777343 ---- Protocol UDP -
4
5                  localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
6  HELO
7          500
8   ERROR DE SINTAXIS
9
10
11
12                 localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
13
14                 localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
15 HOLA
16         250
17  ¿A cómo está hoy la prima de riesgo?#5
18
19
20                 localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
21
22                 localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
23 no lo se
24         500
25  ERROR DE SINTAXIS
26
27
28
29                 localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
30
31                 localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
32 RESPUESTA 67
33         354
34  MAYOR#4
35
36
37
38                 localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
39
40                 localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
41 RESPUESTA 110
42         350
43  ACIERTO
44
45
46
47                 localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
48
49                 localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
50 +
51         250
52  ¿Cuántos estados tiene Estados Unidos?#3
```

Continuación:

```
53
54
55                  localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
56
57                  localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
58 RESPUESTA 51
59          350
60  ACIERTO
61
62
63
64                  localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
65
66                  localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
67
68 ADIOS
69          221
70  Cerrando el servicio
71                  localhost port 40828, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
72 Completed localhost port 40828, 0 requests, IP 16777343, Protocol UDP, at Tue Dec 12 18:39:43 2023
```

ASPECTOS DESTACABLES Y DAEMON:

Para la obtención de las preguntas que hará el servidor al cliente, creamos una función para leerlas de un archivo preguntas.txt, en el que las preguntas se separan de la respuesta por un *.

Después, almacenamos esa información en un struct Datos.

```c
Datos readNextLine(FILE* file)
{
    Datos result;
    char buffer[100];

    if (fgets(buffer, sizeof(buffer), file) != NULL)
    {
        // Usar strtok para dividir la línea en campos separados por '*'
        char *token = strtok(buffer, "*");

        // Copiar el primer campo (pregunta) en result.pregunta
        if (token != NULL)
        {
            strncpy(result.pregunta, token, sizeof(result.pregunta));
            result.pregunta[sizeof(result.pregunta) - 1] = '\0';  // Asegurar terminación nula
        }
        else
        {
            // Si no hay campo, poner result.pregunta en cadena vacía
            result.pregunta[0] = '\0';
        }

        // Obtener el siguiente campo (solucion) usando atoi
        token = strtok(NULL, "*");
        if (token != NULL)
        {
            result.solucion = atoi(token);
        }
        else
        {
            // Si no hay campo, poner result.solucion en 0
            result.solucion = 0;
        }
    }
    else
    {
        // Si no se puede leer más líneas, poner result.pregunta en cadena vacía y result.solucion en 0
        result.pregunta[0] = '\0';
        result.solucion = 0;
    }

    return result;
}
```

```c
typedef struct {
    char pregunta[TAM_BUFFER];
    int solucion;
} Datos;
```

A la hora de comprobar las tramas de respuesta, decidimos de separar las tramas por tokens para poder trabajar mejor con la información que contienen.

```c
tokens = strtok(buff, seps);

int i = 0;
char respuesta[strlen(tokens)][TAM_BUFFER];

while (tokens != NULL)
{
    sprintf(respuesta[i], "%s", tokens);
    tokens = strtok(NULL, seps);
    i++;
}

strcpy(buff, respuesta[0]);
strcat(buff, " ");
strcat(buff, respuesta[1]);
strcat(buff, "\n");
```

Además, comprobamos que el segundo parámetro de las tramas de respuesta sea un dígito para poder hacer un cast y compararlo con la respuesta correcta a la pregunta. Lo hacemos con una función complementaria.

```
if(strcmp(buffTemp,"ADIOS\r\n")==0)
{
    *retorno = 2;
    *temp2 = 2;
    return;
}

if (strcmp(buffTemp, "+\r\n") == 0)
{
    memset(buff, 0, strlen(buff));
    strcpy(buff, "+\r\n");
    *retorno = 5;
    *temp2 = 0;
    return;
}

if(i != 2 || strcmp(respuesta[0], "RESPUESTA") != 0 || esNumero(respuesta[1]) != 1)
{
    memset(buff, 0, strlen(buff));
    strcpy(buff, buffTemp);
    *temp2 = -1;
    *retorno = -1;
    return;
}

else
{
    solucion = atoi(respuesta[1]);

    if (solucion < juego.solucion)
    {
        *retorno = 4;
    }
    else if (solucion > juego.solucion)
    {
        *retorno = 3;
    }
    else if (solucion == juego.solucion)
    {
        *retorno = 0;
    }

    *temp2 = 0;
}
```

```
int esNumero(const char *cadena)
{
    int i = 0;

    // Si el primer caracter es un signo, se ignora
    if (cadena[0] == '+' || cadena[0] == '-')
    {
        i++;
    }

    while (cadena[i] != '\0')
    {
        if (!isdigit(cadena[i]))
        {
            return 0;  // No es un número
        }
        i++;
    }

    return 1;  // Es un número
}
```

Por último, encontramos un fallo al descargar los ficheros de ordenes de diaweb en el que al final de cada línea, acababa con \r\r\n. Al final descubrimos que se trata por la “incompatibilidad” de los ficheros de texto entre sistemas operativos, en este caso Windows y Linux.

Aun así, hasta que descubrimos esto, usamos una función para el debug en la que se mostrarían en consola las tramas con los respectivos finales de línea o retornos de carro.

```
void imprimirConSecuenciasDeEscape(const char *cadena) {
    while (*cadena) {
        if (*cadena == '\n') {
            printf("\\n");
        } else if(*cadena == '\r') {
       printf("\\r");
    }else{
            putchar(*cadena);
        }
        cadena++;
    }
}
```

Además, todo el procesamiento y envío de tramas lo hacemos a través de funciones propias.

Funciones del servidor:

```
//------------------------------------------------------------------------
// PROTOTIPOS DE FUNCIONES GENERALES
//------------------------------------------------------------------------
Datos readNextLine(FILE* file);
int esNumero(const char *cadena);
void imprimirConSecuenciasDeEscape(const char *cadena);
int RecibirTramaAdios(int s, char* buffffff, int* len);


//------------------------------------------------------------------------
// PROTOTIPOS DE FUNCIONES TCP
//------------------------------------------------------------------------
int RecibirTramaHolaTCP(int s, char* buf, int* len, char* hostname);
void RecibirTramaRespuestaTCP(int s, char* buff, int *len,int *retorno, int *temp2, Datos juego);


//------------------------------------------------------------------------
// PROTOTIPOS DE FUNCIONES UDP
//------------------------------------------------------------------------
void RecibirTramaHolaUDP(int s, char* buf, int *temp1, int *len, char* hostname);
void RecibirTramaRespuestaUDP(int s, char* buff, int *len,int *retorno, int *temp2, Datos juego);
```

Funciones del cliente:

```
//------------------------------------------------------------------------
// PROTOTIPOS DE FUNCIONES TCP
//------------------------------------------------------------------------
int EnvioHolaTCP(int s, char* bufHola, char* argv[], char* hostname);
int EnvioRespuestaTCP(int s, char* bufRespuesta);
int EnvioAdiosTCP(int s, char* bufSalida);

//------------------------------------------------------------------------
// PROTOTIPOS DE FUNCIONES UDP
//------------------------------------------------------------------------
int EnvioHolaUDP(int s, char* bufHola, char* argv[], char* hostname);
int EnvioRespuestaUDP(int s, char *bufRespuesta);
int EnvioAdiosUDP(int s, char* bufSalida);
```

Para acabar, se usa el daemon lanzaServidor.sh para su ejecució múltiple e

```
# lanzaServidor.sh
# Lanza el servidor que es un daemon y varios clientes
# las ordenes est�n en un fichero que se pasa como tercer par�metro
./servidor
./cliente nogal TCP ordenes.txt &
./cliente nogal TCP ordenes1.txt &
./cliente nogal TCP ordenes2.txt &
./cliente nogal UDP ordenes.txt &
./cliente nogal UDP ordenes1.txt &
./cliente nogal UDP ordenes2.txt &
```